START HERE

Understand the Connectivity Credits demo

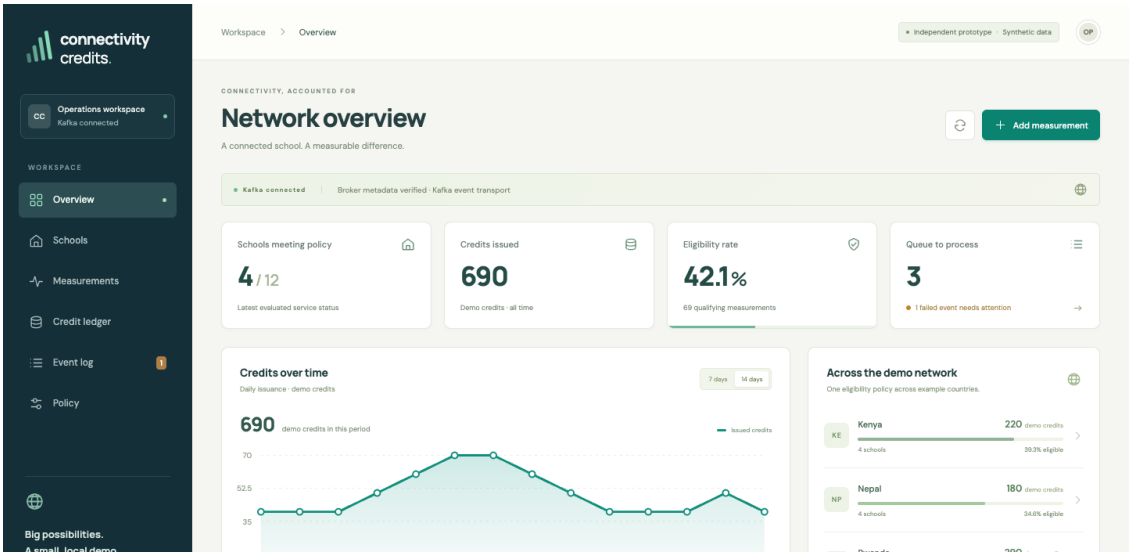
A small working system that checks internet-quality readings and records a demo reward when the rules are met.

I built and hosted this prototype for my application to UNICEF's Connectivity Credits backend consultancy. It gives the hiring team a concrete way to assess how I would approach the backend work.

Open the live demo | View the source on GitHub

The idea in one example

Imagine an internet provider serving a school. The provider becomes eligible for a reward if the connection meets agreed standards. This prototype accepts a fictional reading, checks it against a policy and records **10 demo credits** if it passes. A credit here is only a database entry; no money or token is transferred.



The overview screen. These school counts, charts and credit totals come from fictional test data.

What is working and what is simulated

The website, API, PostgreSQL database and Kafka event processing are running software. The schools, readings, verification results and rewards are simulated. The demo is independent of UNICEF and does not connect to its systems.

Read next: page 2 walks through the buttons; page 3 shows the architecture; page 4 explains processing; page 5 explains the numbers; page 6 maps the technology and skills.

01 / TRY IT

A guided tour in five minutes

Open the live demo and choose **Overview** in the left menu. On a phone, open the menu first. Scroll to **From signal to credit**, where four buttons run prepared examples. Each button creates and processes its own fictional readings.

1. Run clean batch

Click **Run clean batch**. Three readings meet the current policy. The message reports three credit issuances. Open **Credit ledger** to inspect the entries. With the default policy, that means 3 entries worth 10 demo credits each: **30 demo credits** in total.

2. Test below threshold

Return to **Overview** and click **Test below threshold**. A reading opens with its individual checks. Some fail, so that reading earns **no credit**. This is a valid business decision, not a software failure.

3. Replay a delivery

Click **Replay a delivery**. The same submission arrives three times. The result is **one reading, one event and one credit issuance**. The other two deliveries are recognized as duplicates. At the default rate this adds 10 credits, not 30.

4. Simulate worker failure

Click **Simulate worker failure**. The app opens **Event log** with a deliberately failed event. No credit is retained for that attempt. Click **Retry** on the new failed row: processing succeeds and one credit is issued. This shows recovery without a partial or duplicate reward.

Try your own values

Click **Add measurement** to enter fictional values, then **Add to queue**. This saves the input; it does not yet evaluate it. Use **Event log** → **Process queue** to run the checks. Inspect the reading under **Measurements**. Choose a school/date that has no credit yet; a school can receive only one issuance per day.

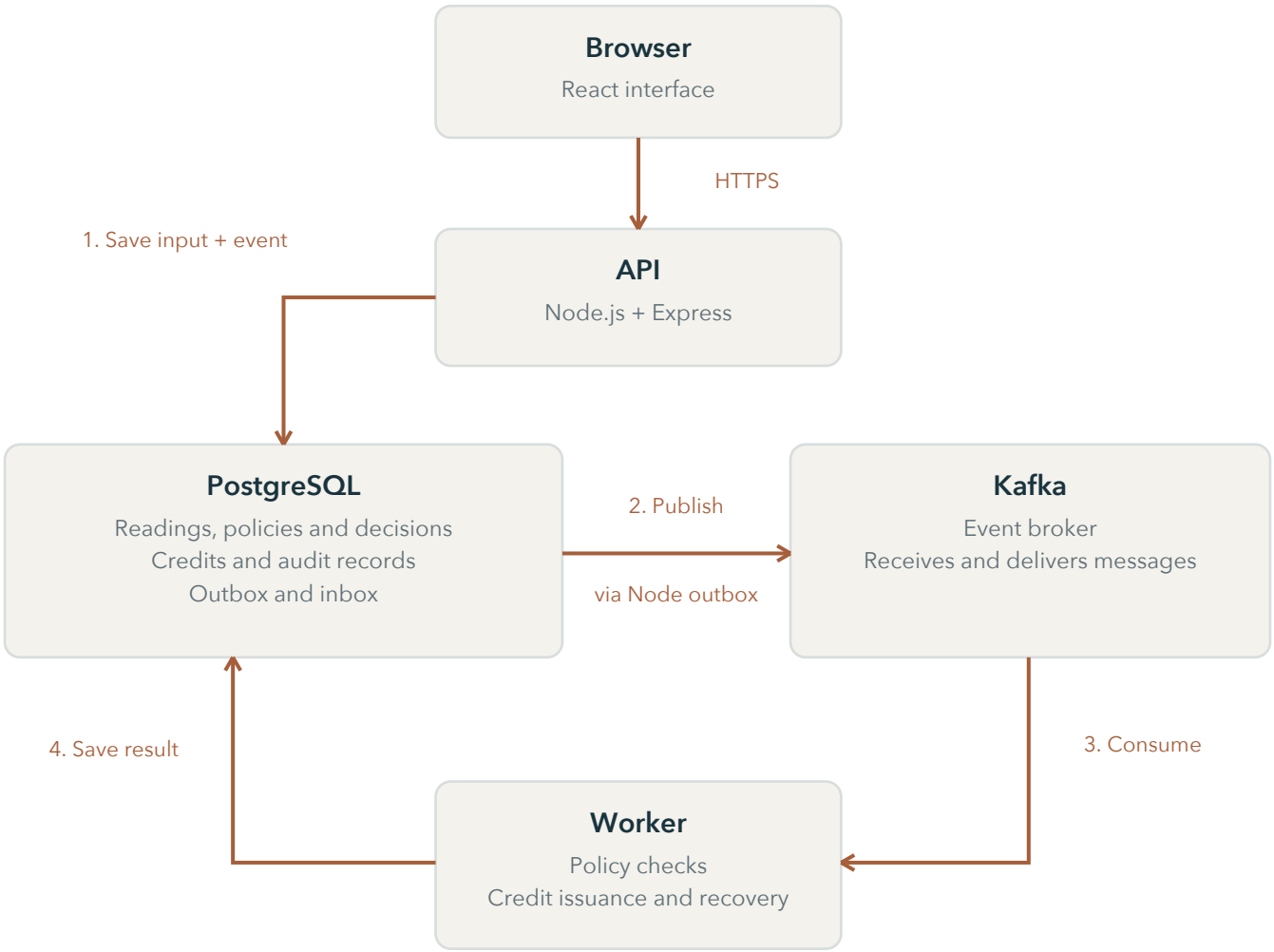
The public demo is shared

Other visitors can change the data and policy. Check the result of your scenario rather than expecting a fixed dashboard total. **Reset demo data** restores the starting examples and policy for everyone, so it also clears other visitors' experiments. The starting data intentionally includes queued and failed work.

02 / ARCHITECTURE

How the running demo fits together

The browser sends actions to the API. PostgreSQL keeps the records; Kafka carries messages; the worker checks the rules and saves the outcome.



The arrows follow one reading. A publisher in the Node application sends saved events to Kafka. The worker reads the policy before saving a decision. The API also returns stored records to the browser.

Three services on the host

The API, publisher and worker are logical components inside the **same Node.js container**. PostgreSQL and Kafka run in separate containers. Coolify hosts all three on Hetzner. The Node app serves the React interface; scenario, process and retry buttons trigger processing.

03 / UNDER THE HOOD

What happens after a reading arrives



1. Receive. The API is the entry point for the browser and other systems. It checks that a reading has valid fields, then saves it in PostgreSQL. It also saves a pending message in the same database operation.

2. Queue. An event is a message saying there is work to do. Kafka carries these messages to the processing code. The database keeps a record of messages awaiting delivery (the **outbox**) and messages received (the **inbox**), so a temporary interruption does not rely on browser memory.

3. Check. The **worker** is the code that does the queued work. It compares the reading with the current **policy**: five conditions that must all pass. It saves the policy version and each check, so a reviewer can understand the decision later.

4. Record. The decision, any credit and the audit entry are saved together. The **ledger** is the list of issued credits. The **audit trail** records what happened and why.

Why the failure and replay buttons matter

Term you may see	What it means here
Idempotency	Repeating the same submission does not create another reward. There is also a limit of one issuance per school and date.
Transaction and rollback	Related database changes succeed together or are undone together. An interrupted issuance leaves no half-finished credit.
Retry	Try failed work again after the problem is resolved. Duplicate protection still applies.
Policy version	A saved copy of the rules used for a decision. Later policy edits do not rewrite earlier decisions.

Kafka may deliver a message more than once. The database protects the credit result. The demo does not claim that every message can be delivered exactly once.

04 / READ THE NUMBERS

What the measurements actually mean

All five checks must pass to issue a credit. These are **illustrative demo defaults**, not UNICEF requirements. You can inspect or change them in **Policy**.

Input	Plain meaning	Default rule
Download	How fast data can reach the school, in megabits per second (Mbps).	At least 20 Mbps
Upload	How fast the school can send data out.	At least 5 Mbps
Latency	The time for a response, in milliseconds (ms). Lower is better.	At most 100 ms
Uptime	The share of the agreed observation period when service was available.	At least 98%
Verification	Whether the reading has a verified label. This label is simulated here.	Verified

Example: 30 Mbps down, 8 Mbps up, 60 ms latency, 99% uptime and verified status pass. For a school/date with no previous issuance, that creates one record worth 10 demo credits.

Why are values entered manually?

The form supplies controlled test inputs. It lets you see what happens when a rule passes or fails. It does **not** perform a speed test. The prepared scenarios and starting dataset also use fictional values.

For real school data, tests would run from the school network and send results automatically through an approved integration. A test in your browser measures your connection; a test from Hetzner measures the server's connection. Neither measures the schools on this screen.

Ping can estimate round-trip delay and response loss; measuring upload/download speed requires transferring data. Uptime requires repeated observations over time. One successful ping cannot establish a daily uptime percentage. Giga Meter already runs automated school-side tests.

How to read the dashboard

Credits issued adds up demo credit amounts. **Schools meeting policy** counts schools whose latest reading passes. **Eligibility rate** shows the share of evaluated readings that qualify. **Queue to process** means saved work awaiting processing. **Kafka connected** means the message broker is reachable; it says nothing about a school's internet being online.

05 / CAPABILITIES

Technology and demonstrated skills

Component	Technology	Role in the demo
Interface	React 18 + TypeScript Vite 6	Screens, forms and scenario controls.
API and worker	Node.js 24 + Express 5 TypeScript + Zod	Validate input, run policy checks and coordinate work.
Database	PostgreSQL 16 pg client	Persist readings, decisions and credits; protect against partial or duplicate results.
Event transport	Apache Kafka 4.4	Carry messages between ingestion and processing.
Hosting and checks	Docker Compose Coolify on Hetzner GitHub Actions	Run the hosted services and build/test the application.

What reviewers can verify

The live demo demonstrates policy decisions, replay protection and recovery from an injected failure. The repository contains automated API, database and Kafka checks, passing CI records, setup instructions and desktop/mobile validation evidence. See the validation record for commands and limits.

Engineering skills on display

The work sample brings together API design and input validation, relational data modelling, asynchronous event processing, explicit business rules, transaction handling and recovery. The console makes those backend decisions inspectable, while the tests exercise duplicate submissions, failed processing and policy history.

How it relates to the consultancy

The supplied terms of reference call for backend services, event handling, rules, external integrations and operational reliability. This prototype demonstrates part of that work. TypeScript is within the listed language requirements and Kafka is named in a deliverable. Express, React and PostgreSQL are choices made for this work sample.

Links and references

Live demo | GitHub source and setup
UNICEF vacancy 596538 and the supplied consultancy terms of reference.
Giga Meter documentation explains automatic collection. M-Lab NDT documentation explains throughput testing.